Badge Overlay

User Guide · Version 2026.09.08.0001

A floating badge that names the UI element under your cursor -- in any app, on any button, in real time. Designed for low-vision users, screen-reader supplementers, accessibility auditors, and anyone who wants a second set of eyes on what they're about to click.

Badge Overlay works alongside your existing cursor -- including YoloMouse, Windows custom cursor schemes, and third-party cursor hooks. It never replaces or fights with your cursor.

Screenshots

> Note: screenshots are placeholder paths. Add actual PNGs to `docs/screenshots/` and the README will display them.

![Main window -- light theme](docs/screenshots/main-light.png)

![Main window -- dark theme](docs/screenshots/main-dark.png)

![Badge in flight -- labelling a Chrome toolbar button](docs/screenshots/badge-chrome.png)

![Info Overlay -- large-text side panel](docs/screenshots/info-overlay.png)

![Badge Colours dialog -- live WCAG contrast check](docs/screenshots/badge-colours.png)

Features

- **Always-on labelling** -- a compact pill follows your pointer and names the button, icon, tab, link, or control beneath it.
- **Works everywhere** -- Win32 apps, WPF, WinForms, UWP/WinRT, Office ribbon, Chromium browsers (Chrome, Edge, Brave, Vivaldi, Opera), Firefox, Electron apps, File Explorer, desktop icons, taskbar, Start menu, system tray.
- **Info Overlay** -- optional large-text side panel for low-vision users (12 pt to 72 pt, fully keyboard-draggable).
- **Custom rules** -- teach the detector your own button names, by class or by visible text.
- **Category colours** -- eight semantic categories (Close, Minimise, Maximise, Action, Danger, Nav, Browser, Control) with live WCAG AAA contrast verification.
- **Hotkeys** -- global toggle, badge freeze, copy current label.
- **Single cursor-class trick** -- Badge Overlay clears its window class cursor and suppresses client-area `WM_SETCURSOR`, so YoloMouse and custom cursor schemes keep working the way you expect.
- **First-run welcome tour** -- three-screen orientation; replayable from About.
- **F1 cheat-sheet** -- every keyboard shortcut in one window, generated from your current bindings.
- **High-contrast Windows theme support** -- auto-applied; honours `SystemParameters.HighContrast`.

Feature matrix

Capability	Status
Win32 button detection	[done] via MSAA + class
WPF / UWP detection	[done] via UI Automation

Capability	Status
Chromium browser toolbars / tabs	[done]
Firefox toolbar	[done]
Office ribbon	[done]
Electron apps	[done]
File Explorer / Desktop / Taskbar	[done]
Custom rules (text / class match)	[done]
Per-category colour customisation	[done]
WCAG AAA contrast on every theme	[done]
High-Contrast Windows theme follow	[done]
Reduce-motion preference honoured	[done]
Screen-reader live-region announcements	[done]
Global hotkeys (toggle / freeze / copy)	[done]
Per-application profiles	planned
Color-blind preset palettes	planned
Voice readout	planned
Multi-monitor profile	planned

How it differs from YoloMouse

YoloMouse and Badge Overlay solve adjacent problems and **work well together**:

Concern	YoloMouse	Badge Overlay
Primary purpose	Make the cursor easier to *see*	Make the element under the cursor easier to *understand*
Mechanism	Replaces the OS cursor sprite	Floats a label near the cursor (cursor untouched)
Touches your cursor?	Yes, by design	**No.** Badge Overlay deliberately preserves whatever cursor you've configured
Label text	None -- visual only	Reads the underlying control name (button text, accessible name, custom rule)
Accessibility integration	None	UI Automation, MSAA, custom rule engine
WCAG-tested	n/a	Yes -- AAA contrast, focus indicators, live regions

If you use YoloMouse, install Badge Overlay alongside it. The two complement each other and don't conflict.

WCAG 2.2 AAA

Badge Overlay targets WCAG 2.2 Level AAA where reasonable for a desktop accessibility tool:

- **Contrast (1.4.6 AAA)** -- 7:1 for body text, 4.5:1 for large text; user-picked badge colours get a live ratio check in the Badge Colours dialog.
- **Non-text contrast (1.4.11)** -- focus rings (Cyan on dark, Gold on light) clear 3:1 against any surface; sliders use 3 px stroke + drop-shadow halo for visibility on coloured tracks.
- **Visible focus (2.4.7) / Focus appearance (2.4.11 AAA)** -- every interactive element has a visible keyboard focus indicator distinct from hover.
- **Touch / click targets (2.5.5 AAA)** -- 44×44 px on primary nav and action buttons; dense settings panels use 32-36 px (documented deviation).
- **Status messages (4.1.3)** -- `AutomationProperties.LiveSetting="Polite"` and `RaiseAutomationEvent(LiveRegionChanged)` on state changes; screen readers announce hotkey-driven enable/disable.
- **High-contrast mode (1.4.8)** -- detected automatically via `SystemParameters.HighContrast` and `SystemEvents.UserPreferenceChanged`; swaps to a system-colour-bound theme.
- **Keyboard-only operation (2.1.1)** -- arrow-key drag on the Info Overlay (Shift = 1 px fine, Ctrl = 100 px coarse, Ctrl+Home = centre). Logical focus order, visible focus indicators, F1 shortcut reference.
- **Tooltip persistence (1.4.13)** -- 60 s show duration, dismissable via cursor movement.
- **No flashing (2.3.1)** -- no animation over 3 Hz.
- **Reduced motion (2.3.3)** -- honours `SystemParameters.ClientAreaAnimation`; theme cross-fade and other transitions are disabled when reduce-motion is set.
- **Authentication** -- none required. No accounts. No cognitive tests.
- **Assistive-technology identity** -- every interactive control in the settings UI has `AutomationProperties.Name` and `HelpText`; sliders expose `LiveSetting="Polite"` for screen-reader value announcements.

Known accessibility deviations

- **Compact dialog density** -- settings panels use 32-36 px button heights instead of the AAA-recommended 44 px to fit dense option groups in a single window. Primary nav and action buttons are 44 px.
- **Resize handle** -- bottom-right resize triangle is mouse-only. Use Win+Up / Win+Down / `WindowState` shortcuts instead. (Planned: keyboard resize.)

Platform support

- Windows 10 21H2 or later, Windows 11
- x64 only (arm64 build planned)
- .NET 10

Install

Run the NSIS installer (`BadgeOverlay-<version>-setup.exe`) from the latest release. Per-user install; no admin rights required.

Privacy

Badge Overlay does not collect, store, or transmit any usage data. All settings save locally to `%APPDATA%\BadgeOverlay`. No accounts, no telemetry, no network calls except when you explicitly open a link from the About dialog.

This statement is also visible in-app via Help -> About -> Privacy.

Build from source

```
build.bat
```

The batch script assumes `dotnet` is on `PATH`. Outputs an unsigned installer into `bin\Release\` -- no code-signing certificate exists for this app (or any app in this suite), so builds are unsigned by design. SmartScreen will warn on first run; that's expected.

Settings

All user preferences live at `%APPDATA%\BadgeOverlay\settings.json`. Custom rules, badge colours, window geometry, and hotkey bindings persist across sessions. Delete the file to reset to factory defaults.

Crash logs write to `%APPDATA%\BadgeOverlay\crash.log` (rolling, capped at 512 KB). A `.old` backup is kept. Crash logs are local-only -- they are never transmitted.

Compatibility with cursor tools

Badge Overlay is explicitly designed **not to interfere** with cursor customisation tools. Tested against:

- YoloMouse
- Windows 10/11 built-in cursor schemes
- Custom `.cur` / `.ani` cursors via `SetSystemCursor`

If you find a cursor tool Badge Overlay conflicts with, please open an issue.

Licence

See LICENSE.

Contributing

Issues and PRs welcome at the project repository. Please include:

- Windows version (`winver`)
- Badge Overlay version (visible in the About dialog)
- A description of what you expected vs. what happened
- If relevant, the tail of `%APPDATA%\BadgeOverlay\crash.log`